



**BIG DATA**

---

HADOOP

# CHALLENGE OF BIG DATA MANAGEMENT AND ANALYTICS (1)

- Existing DB technology is not prepared for the huge volume
  - Until 2007, Facebook had a 15TB data warehouse by a big-DBMS-vendor
  - Now, ~70TB compressed data added into Facebook data warehouse every day (4x total capacity of its data warehouse in 2007)
  - Commercial parallel DBs rarely have 100+ nodes
  - Yahoo!'s Hadoop cluster has 4000+ nodes; Facebook's data warehouse has 2750+ nodes
- Typical science and medical research examples:
  - Large Hadron Collider at CERN generates over 15 PB of data per year
  - Pathology Analytical Imaging Standards databases at Emory reaches 7TB, going to PB
  - LANL Turbulence Simulation: processing the amount of data at PB level.



# CHALLENGE OF BIG DATA MANAGEMENT AND ANALYTICS (2)

- Big data is about all kinds of data
  - Online services (social networks, retailers ...) focus on big data of online and off-line **click-stream** for deep analytics
  - **Medical image** analytics are crucial to both biomedical research and clinical diagnosis
- Complex analytics to gain deep insights from big data
  - Data mining
  - Pattern recognition
  - Data fusion and integration
  - Time series analysis
  - **Goal:** gain deep insights and new knowledge

# CHALLENGE OF BIG DATA MANAGEMENT AND ANALYTICS (3-4)

- Conventional database business model is not affordable
  - Expensive software license
  - High maintenance fees even for open source DBs
  - Store and manage data in a system at least \$10,000/TB\*
  - In contrast, Hadoop-like systems only cost \$1,500/TB\*\*
- Conventional database processing model is “scale-up” based
  - Performance improvement relies on CPU/memory/storage/network updates in a dedicated site (BSP model, CACM, 1990)
  - Big data processing model is “scale-out” based (DOT model, SOCC’11): relies on continuously adding low cost computing and storage nodes in a distributed way

**MapReduce programming model becomes an effective data processing engine for big data analytics**

# WHY MAPREDUCE?

- A simple but effective programming model designed to process huge volumes of data concurrently
- Two unique properties
  - Minimum dependency among tasks (almost sharing nothing)
  - Simple task operations in each node (low cost machines are sufficient)
- Two strong merits for big data analytics
  - Scalability (Amadal's Law): increase throughput by increasing # of nodes
  - Fault-tolerance (quick and low cost recovery of the failures of tasks)
- Hadoop is the most widely used implementation of MapReduce
  - in hundreds of society-dependent corporations/organizations for big data analytics: AOL, Baidu, EBay, Facebook, IBM, NY Times, Yahoo! ....

# TWO CRITICAL CHALLENGES IN PRODUCTION SYSTEMS

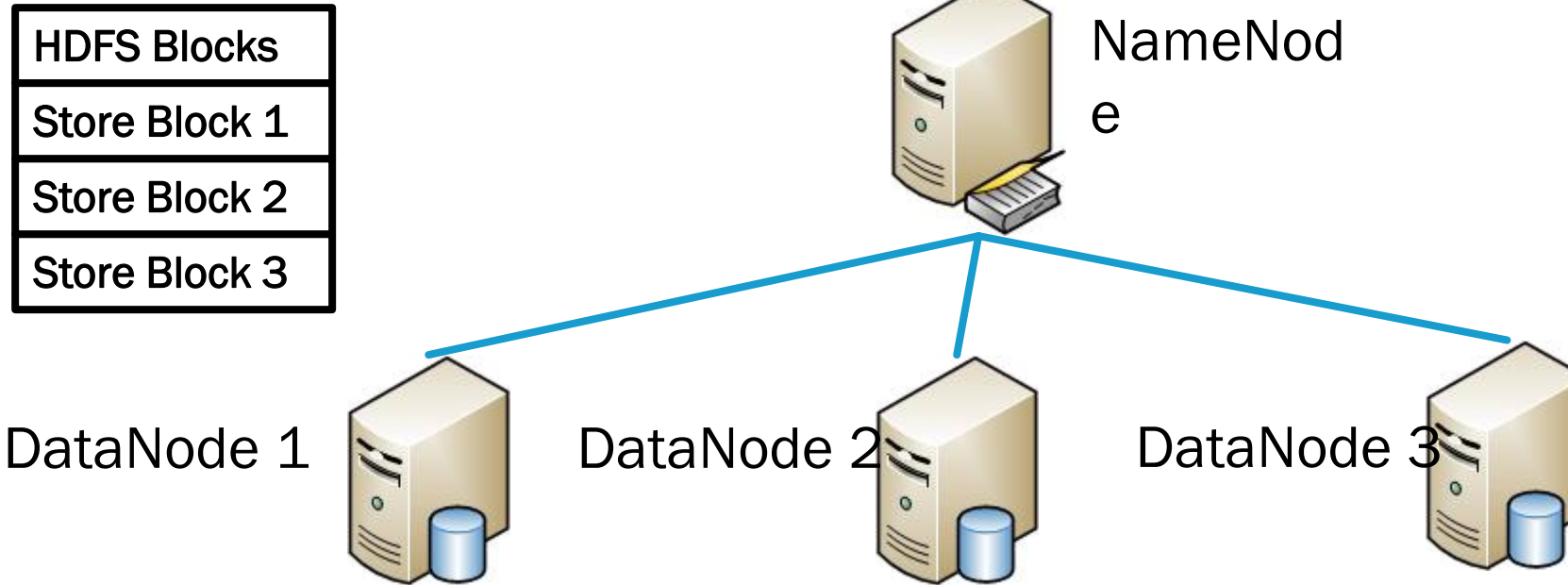
- Background: Standard Relational Databases have been moved to MapReduce Environment, such as Hive and Pig by Facebook and Yahoo!
- Challenge 1: How to initially store big data in distributed systems
  - Objective: to minimize network and storage costs for massive accesses
- Challenge 2: How to automatically convert relational database queries into MapReduce jobs
  - Objectives: to minimize network and storage costs for MR job execution
- Addressing these two Challenges, we aim to achieve
  - High performance of big data analytics
  - High productivity of big data analytics

# CHALLENGE 1: FAST AND STORAGE-EFFICIENT DATA PLACEMENT

- Data loading (L)
  - the overhead of writing data to distributed files ystem and local disks
- Query processing (P)
  - local storage bandwidths of query processing
  - the amount of network transfers
- Storage space utilization (S)
  - Data compression ratio
  - The convenience of applying efficient compression algorithms
- Adaptivity to dynamic workload patterns (W)
  - Additional overhead on certain queries

➤Objective: to design and implement a data placement structure meeting these requirements in MapReduce-based data warehouses

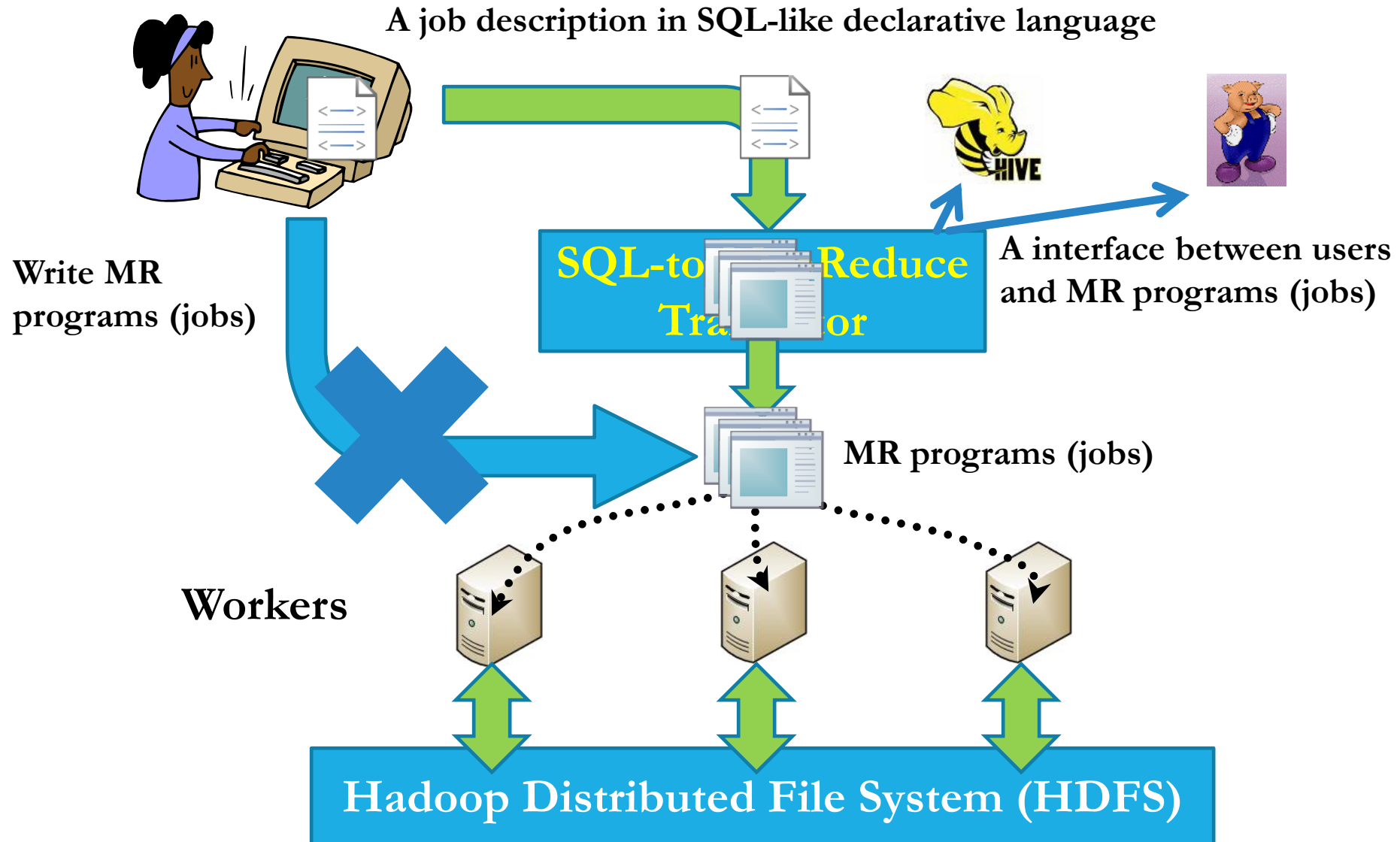
# INITIAL STORES OF BIG DATA IN DISTRIBUTED ENVIRONMENT



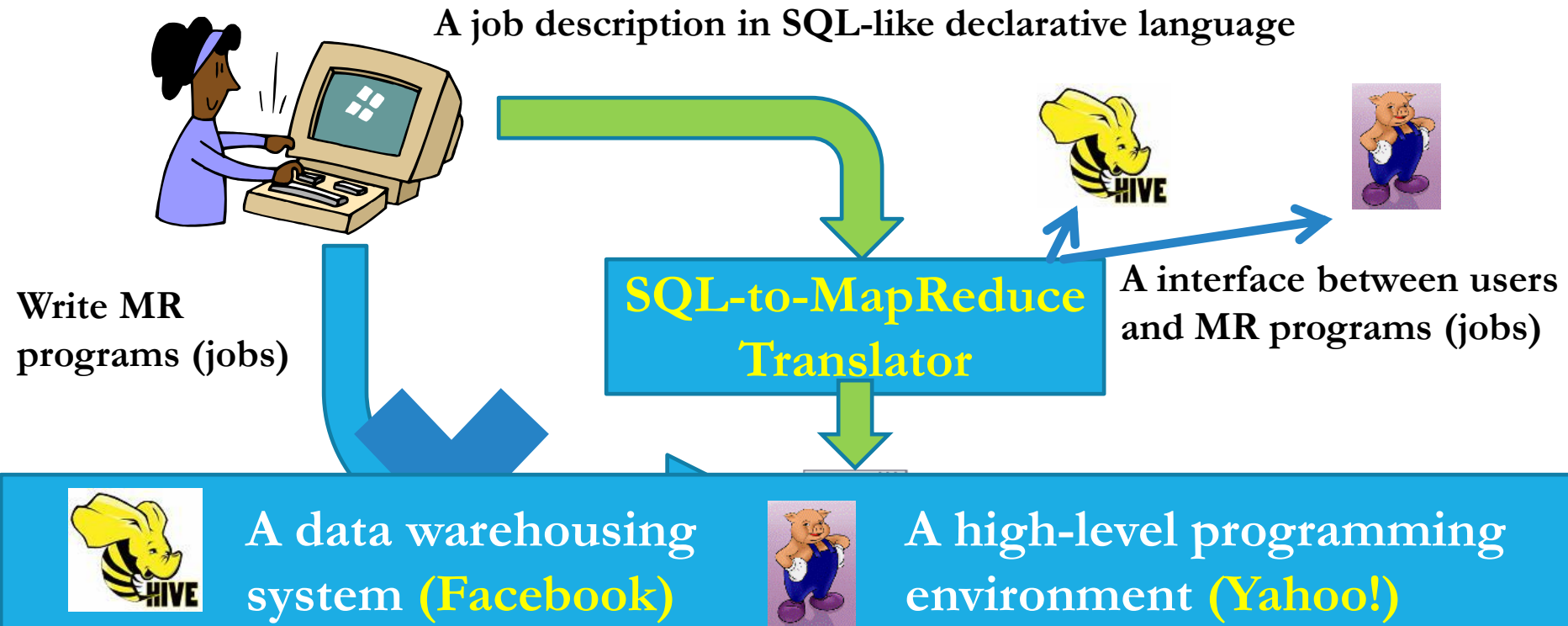
- HDFS (Hadoop Distributed File System) blocks are **distributed**
- Users have a **limited ability to specify** customized data placement policy
  - e.g. to specify which blocks should be co-located
- Minimizing I/O costs in local disks and **intra network communication**



# CHALLENGE 2: HIGH QUALITY MAPREDUCE IN AUTOMATION



## CHALLENGE 2: HIGH QUALITY MAPREDUCE IN AUTOMATION

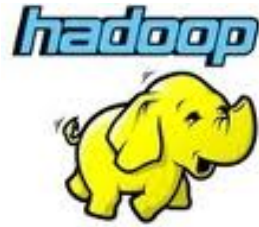


Improve productivity from hand-coding MapReduce programs

- **95%+** Hadoop jobs in Facebook are generated by **Hive**
- **75%+** Hadoop jobs in Yahoo! are invoked by **Pig**\*

# Large-Scale Data Analytics

- MapReduce computing paradigm (E.g., Hadoop) vs. Traditional database systems



vs.



- **Many enterprises are turning to Hadoop**
  - Especially applications generating *big data*
  - Web applications, social networks, scientific applications

# Why Hadoop is able to compete?



VS.



Scalability (petabytes of data, thousands of machines)



Flexibility in accepting all data formats (no schema)



Efficient and simple fault-tolerant mechanism



Commodity inexpensive hardware



Performance (tons of indexing, tuning, data organization tech.)



Features:

- Provenance tracking
- Annotation management
- ....

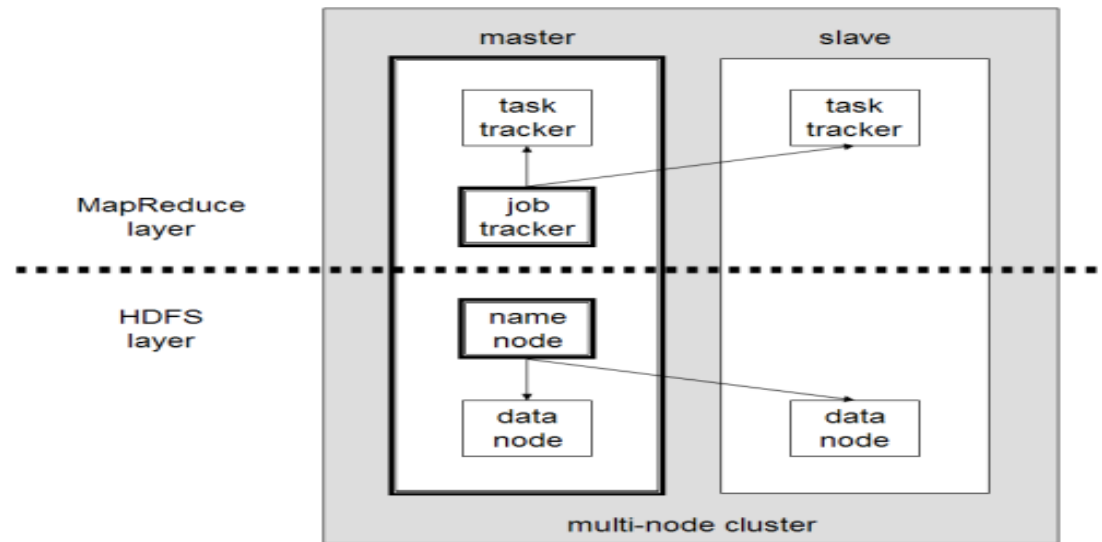
# What is Hadoop

- Hadoop is a software framework for *distributed processing* of *large datasets* across *large clusters* of computers
  - *Large datasets* → Terabytes or petabytes of data
  - *Large clusters* → hundreds or thousands of nodes
- Hadoop is open-source implementation for Google **MapReduce**
- Hadoop is based on a simple programming model called *MapReduce*
- Hadoop is based on a simple data model, *any data will fit*



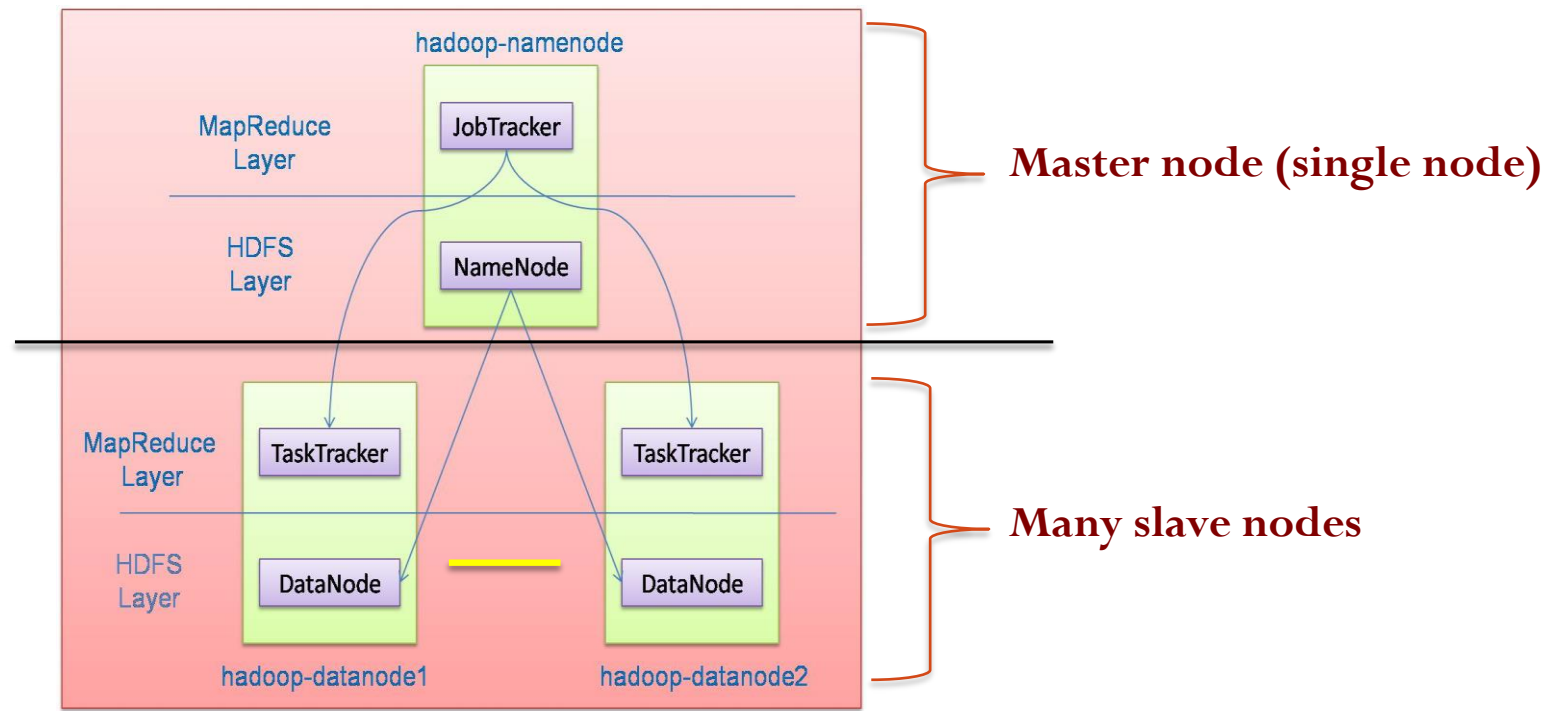
# What is Hadoop (Cont'd)

- **Hadoop framework consists on two main layers**
  - Distributed file system (HDFS)
  - Execution engine (MapReduce)



# Hadoop Master/Slave Architecture

- Hadoop is designed as a *master-slave shared-nothing* architecture



# Design Principles of Hadoop

- Need to process big data
- Need to parallelize computation across thousands of nodes
- **Commodity hardware**
  - Large number of low-end cheap machines working in parallel to solve a computing problem
- This is in contrast to **Parallel DBs**
  - Small number of high-end expensive machines

# Design Principles of Hadoop

- **Automatic parallelization & distribution**
  - Hidden from the end-user
- **Fault tolerance and automatic recovery**
  - Nodes/tasks will fail and will recover automatically
- **Clean and simple programming abstraction**
  - Users only provide two functions “map” and “reduce”

# How Uses MapReduce/Hadoop

- Google: Inventors of MapReduce computing paradigm
- Yahoo: Developing Hadoop open-source of MapReduce
- IBM, Microsoft, Oracle
- Facebook, Amazon, AOL, NetFlex
- Many others + universities and research labs

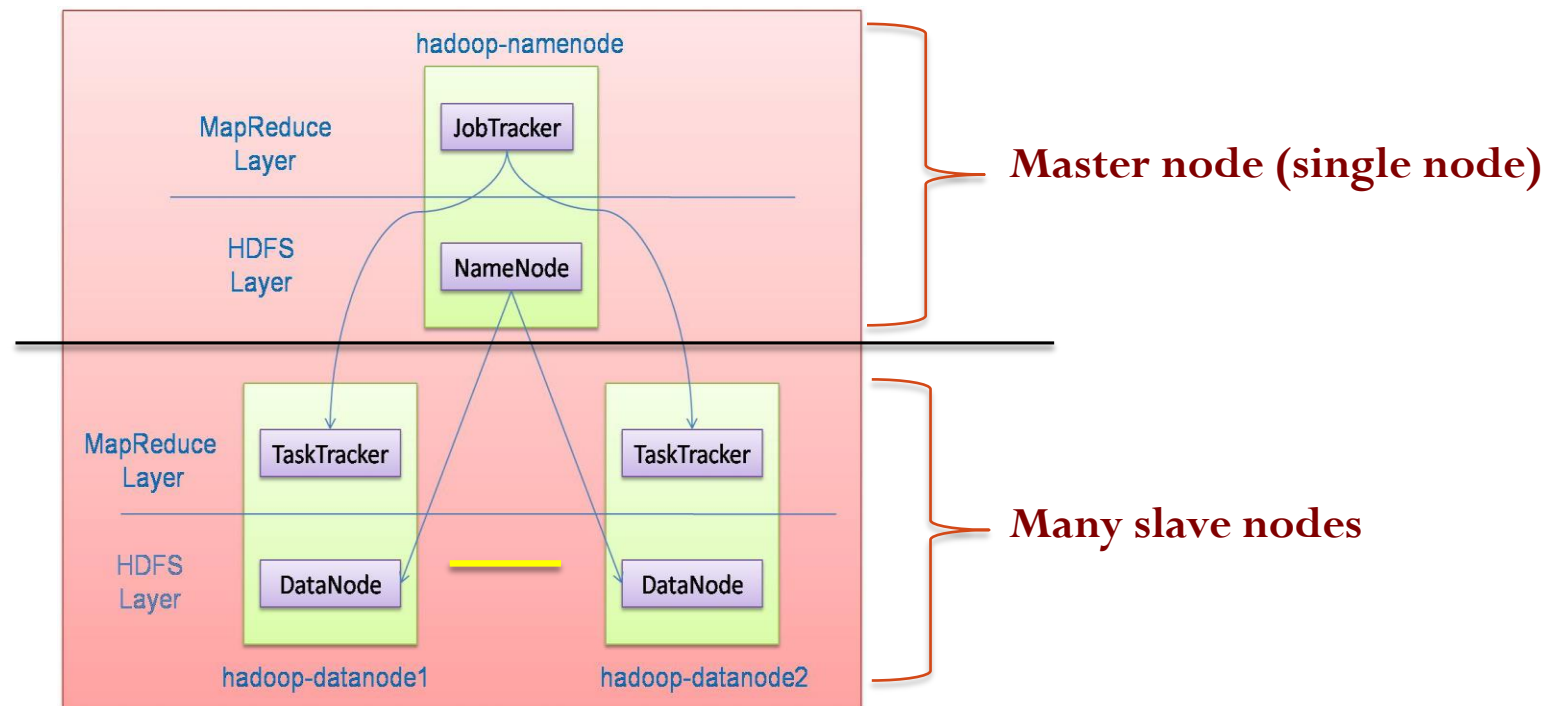
A decorative dashed line in a light brown color curves along the top-left edge of the white circle. A solid gray circle is positioned at the bottom-right edge of the white circle.

# Hadoop: How it Works

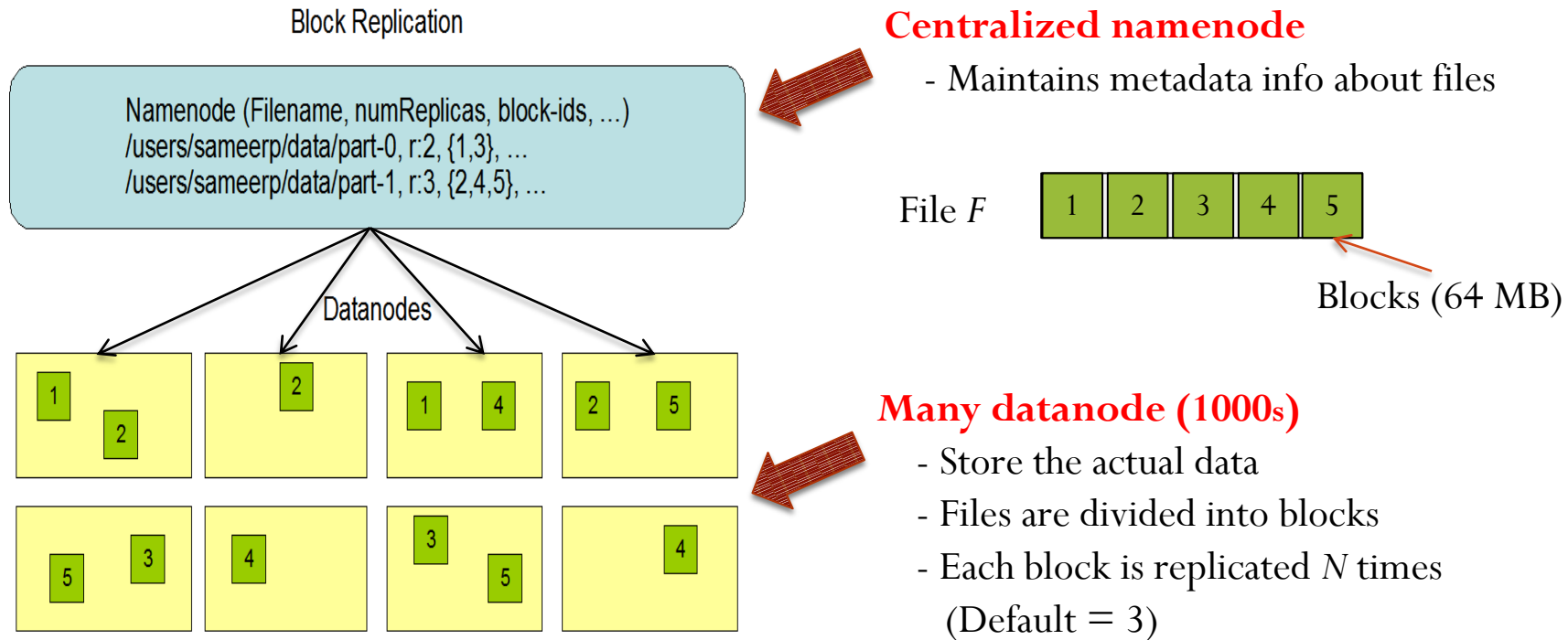


# Hadoop Architecture

- Distributed file system (HDFS)
- Execution engine (MapReduce)



# Hadoop Distributed File System (HDFS)

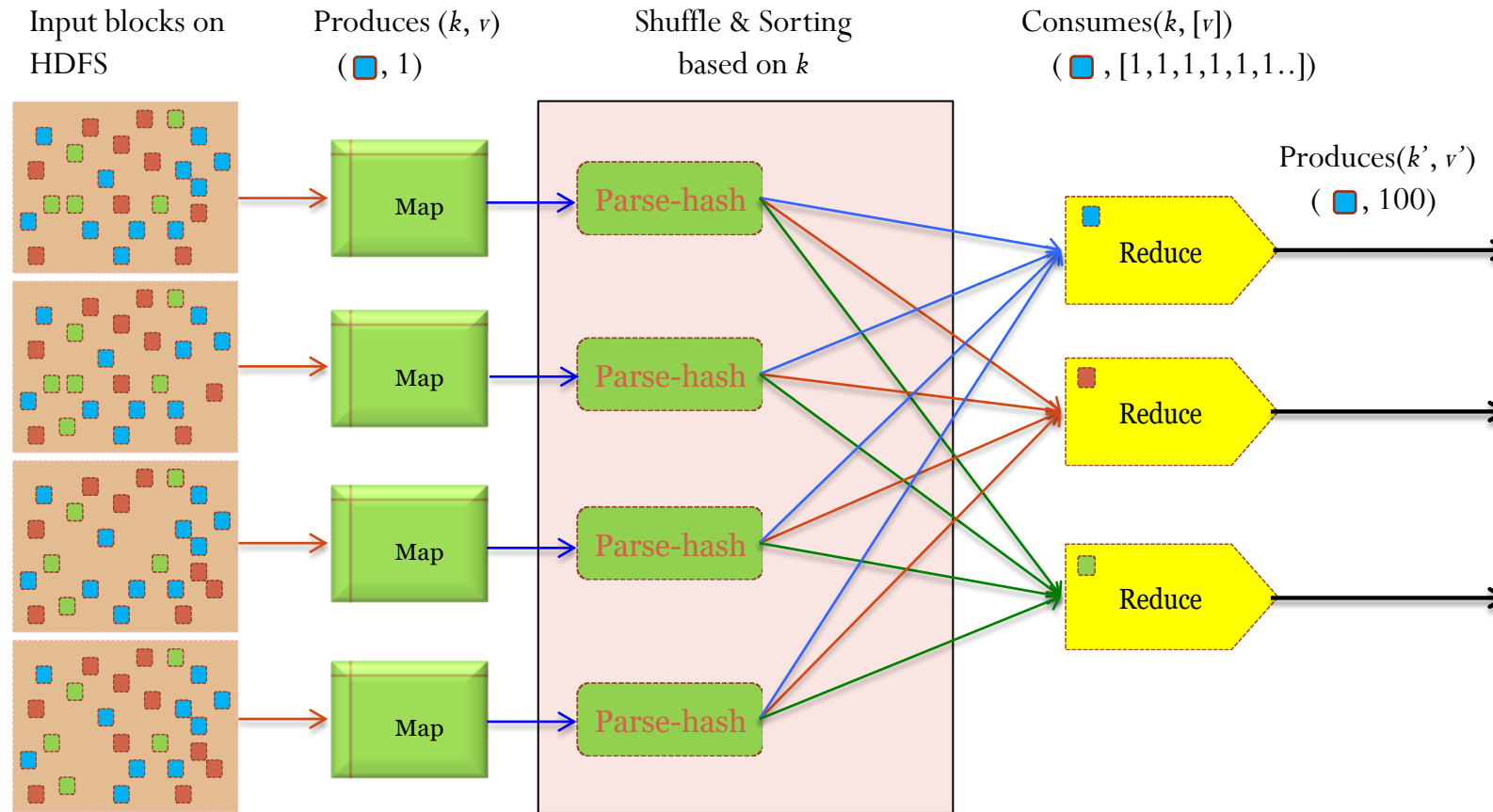


# Main Properties of HDFS

- ***Large:*** A HDFS instance may consist of thousands of server machines, each storing part of the file system's data
- ***Replication:*** Each data block is replicated many times (default is 3)
- ***Failure:*** Failure is the norm rather than exception
- ***Fault Tolerance:*** Detection of faults and quick, automatic recovery from them is a core architectural goal of HDFS
  - Namenode is consistently checking Datanodes

# Map-Reduce Execution Engine

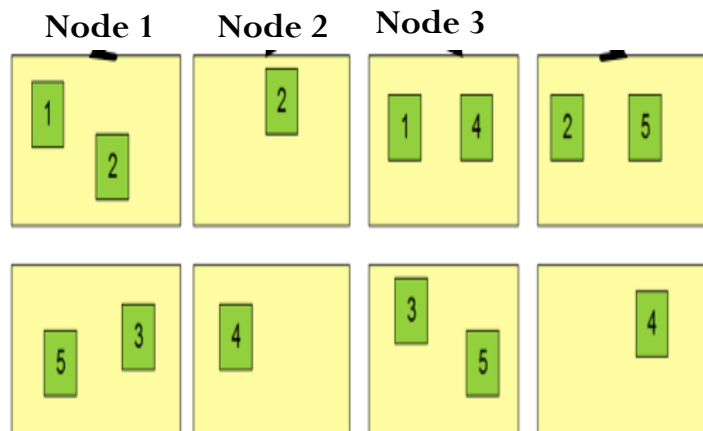
## (Example: Color Count)



*Users only provide the “Map” and “Reduce” functions*

# Properties of MapReduce Engine

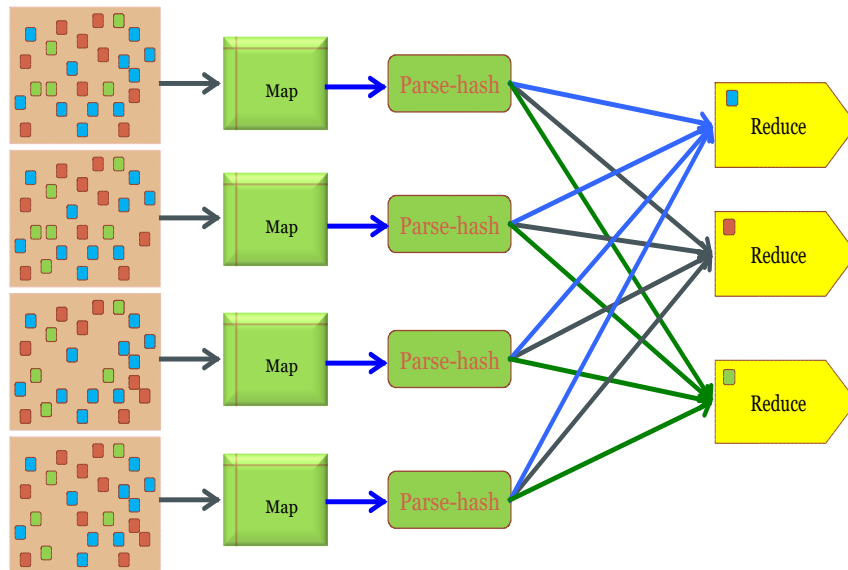
- **Job Tracker is the master node (runs with the namenode)**
  - Receives the user's job
  - Decides on how many tasks will run (number of mappers)
  - Decides on where to run each mapper (concept of locality)



- This file has 5 Blocks → run 5 map tasks
- Where to run the task reading block “1”
  - *Try to run it on Node 1 or Node 3*

# Properties of MapReduce Engine (Cont'd)

- **Task Tracker is the slave node (runs on each datanode)**
  - Receives the task from Job Tracker
  - Runs the task until completion (either map or reduce task)
  - Always in communication with the Job Tracker reporting progress

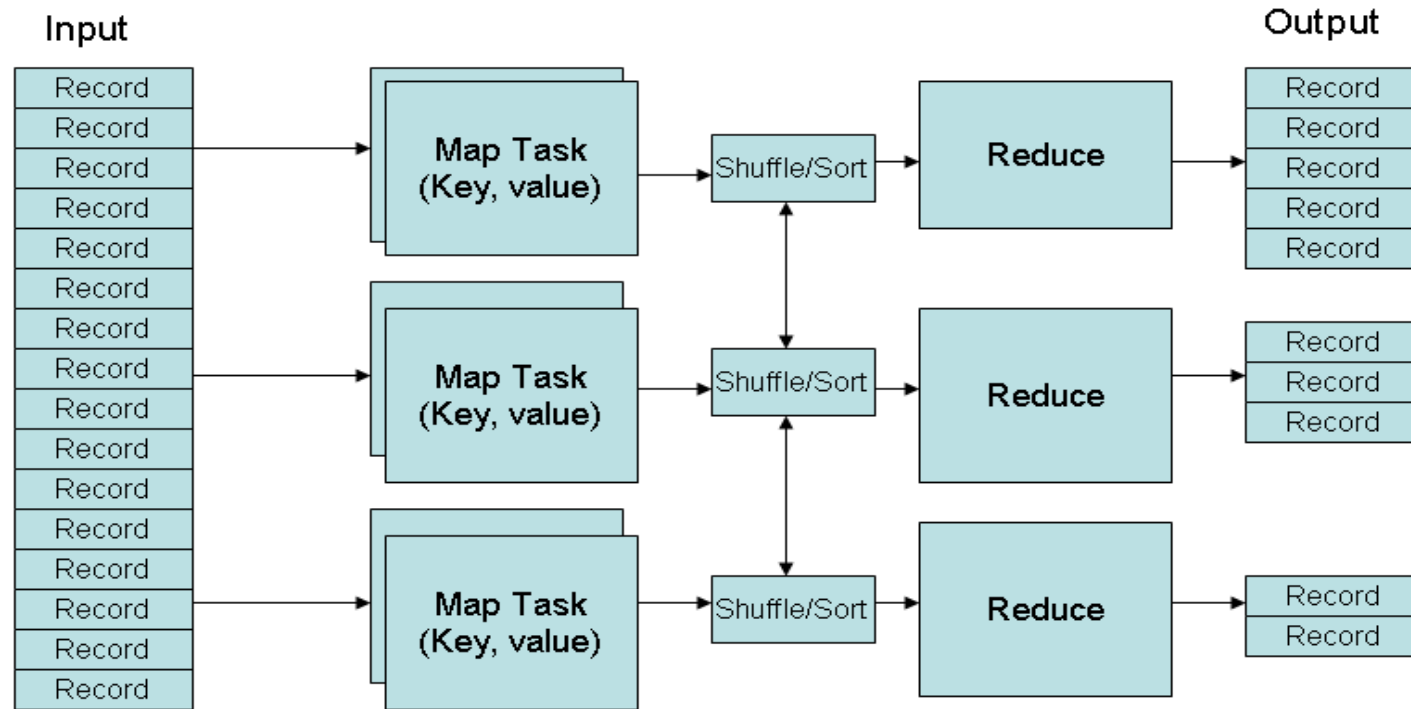


*In this example, 1 map-reduce job consists of 4 map tasks and 3 reduce tasks*

# Key-Value Pairs

- Mappers and Reducers are users' code (provided functions)
- Just need to obey the Key-Value pairs interface
- **Mappers:**
  - Consume <key, value> pairs
  - Produce <key, value> pairs
- **Reducers:**
  - Consume <key, <list of values>>
  - Produce <key, value>
- **Shuffling and Sorting:**
  - Hidden phase between mappers and reducers
  - Groups all similar keys from all mappers, sorts and passes them to a certain reducer in the form of <key, <list of values>>

# MapReduce Phases

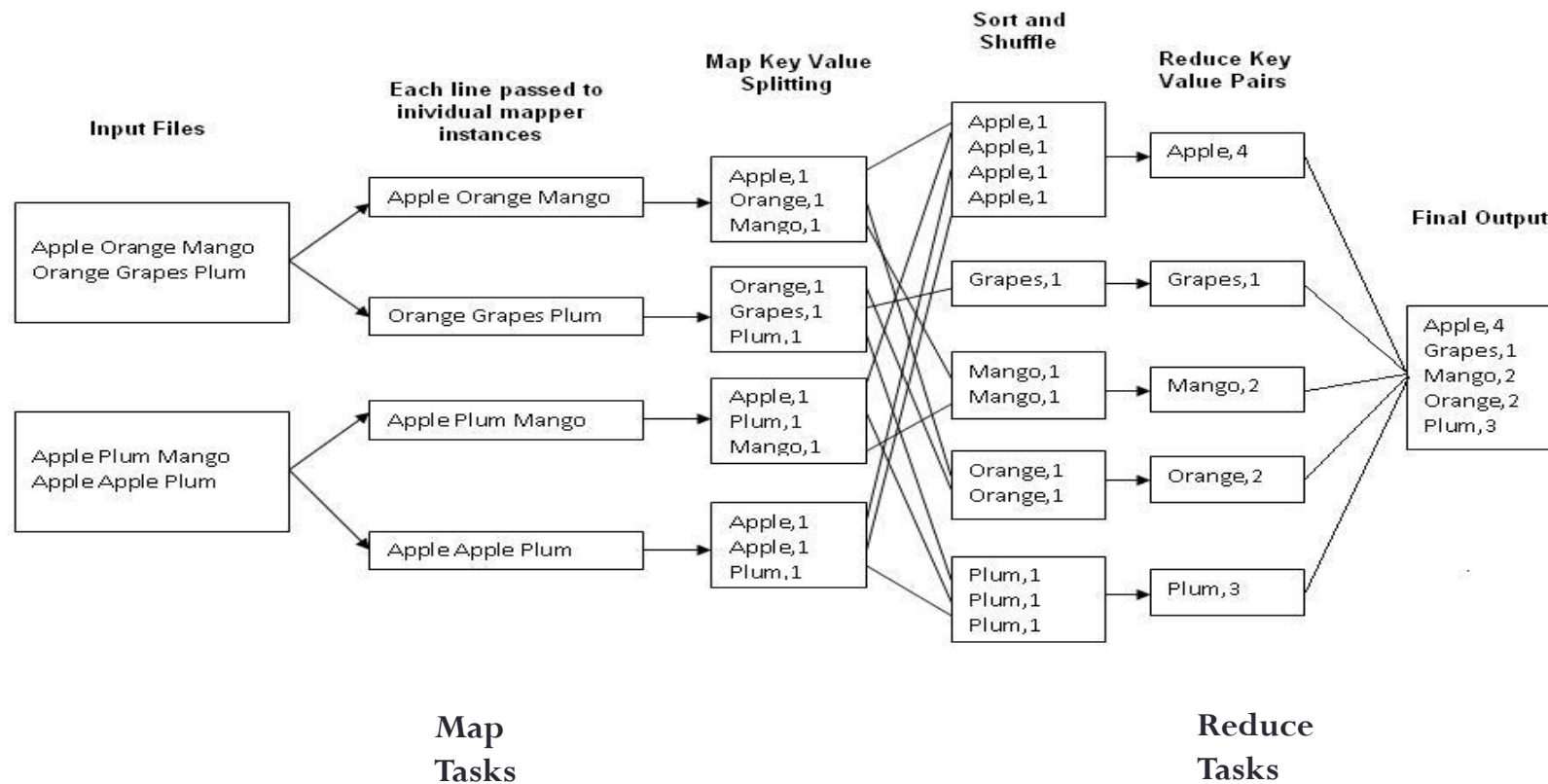


*Deciding on what will be the **key** and what will be the **value** → developer's responsibility*



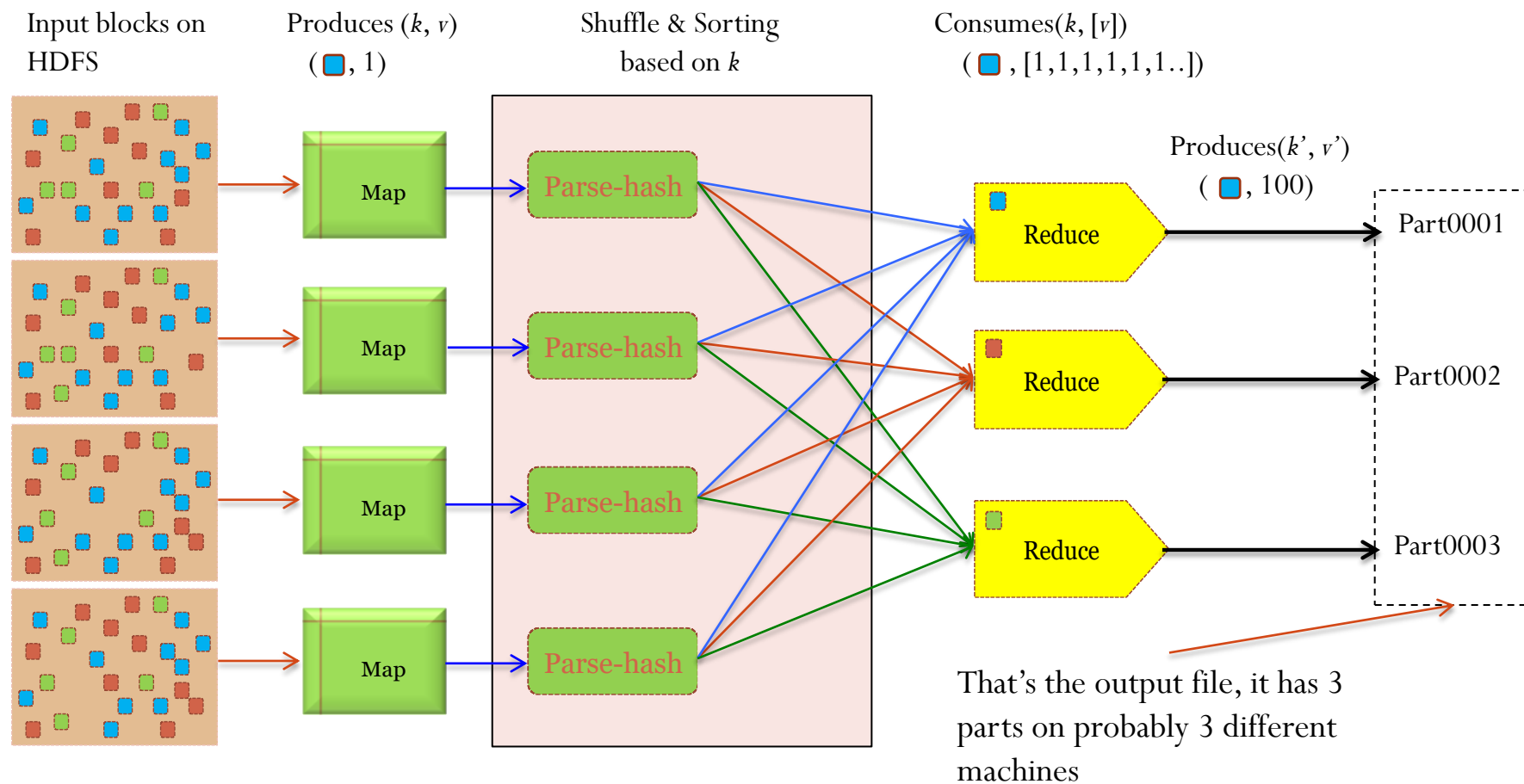
# Example 1: Word Count

- **Job: Count the occurrences of each word in a data set**



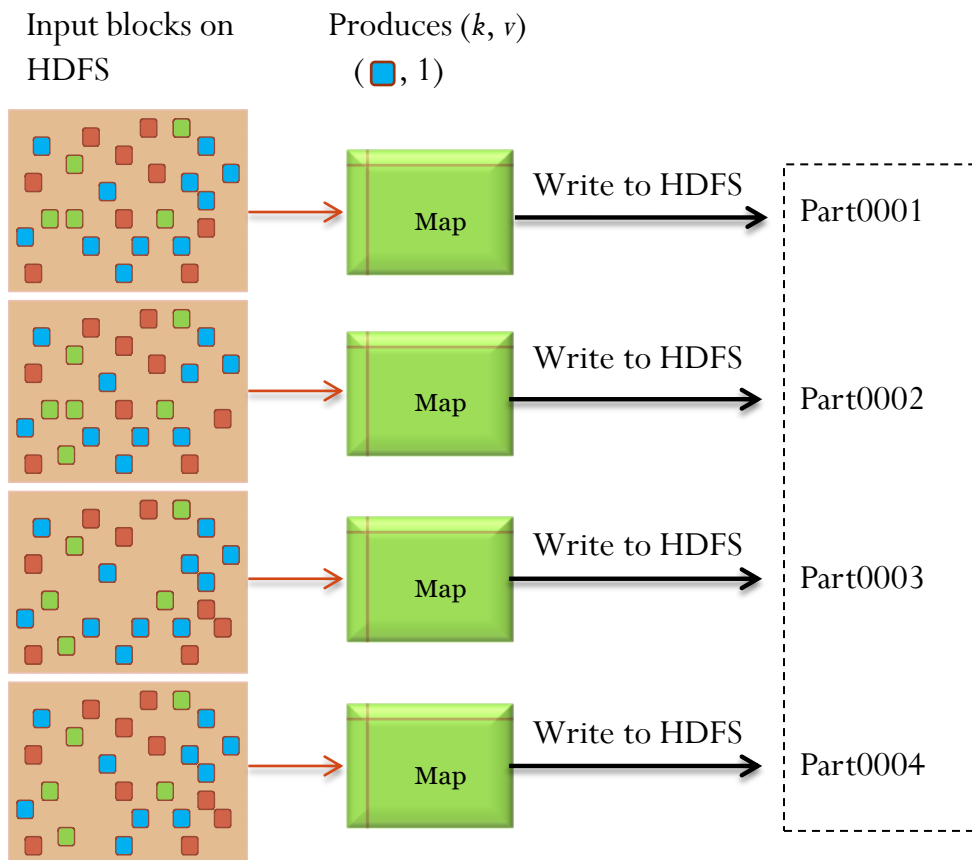
# Example 2: Color Count

**Job: Count the number of each color in a data set**



# Example 3: Color Filter

**Job: Select only the blue and the green colors**



- Each map task will select only the blue or green colors
- No need for reduce phase

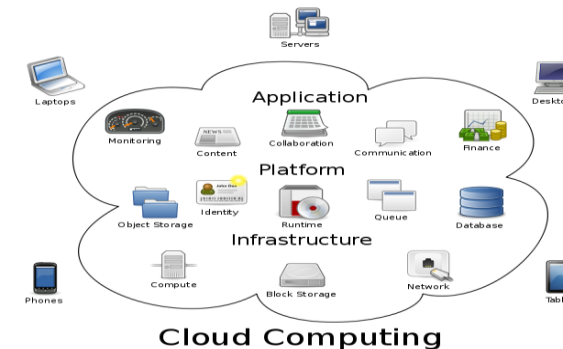
That's the output file, it has 4 parts on probably 4 different machines

# Bigger Picture: Hadoop vs. Other Systems

|                            | Distributed Databases                                                                                                                                                 | Hadoop                                                                                                                                             |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Computing Model</b>     | <ul style="list-style-type: none"> <li>- Notion of transactions</li> <li>- Transaction is the unit of work</li> <li>- ACID properties, Concurrency control</li> </ul> | <ul style="list-style-type: none"> <li>- Notion of jobs</li> <li>- Job is the unit of work</li> <li>- No concurrency control</li> </ul>            |
| <b>Data Model</b>          | <ul style="list-style-type: none"> <li>- Structured data with known schema</li> <li>- Read/Write mode</li> </ul>                                                      | <ul style="list-style-type: none"> <li>- Any data will fit in any format</li> <li>- (un)(semi)structured</li> <li>- ReadOnly mode</li> </ul>       |
| <b>Cost Model</b>          | <ul style="list-style-type: none"> <li>- Expensive servers</li> </ul>                                                                                                 | <ul style="list-style-type: none"> <li>- Cheap commodity machines</li> </ul>                                                                       |
| <b>Fault Tolerance</b>     | <ul style="list-style-type: none"> <li>- Failures are rare</li> <li>- Recovery mechanisms</li> </ul>                                                                  | <ul style="list-style-type: none"> <li>- Failures are common over thousands of machines</li> <li>- Simple yet efficient fault tolerance</li> </ul> |
| <b>Key Characteristics</b> | <ul style="list-style-type: none"> <li>- Efficiency, optimizations, fine-tuning</li> </ul>                                                                            | <ul style="list-style-type: none"> <li>- Scalability, flexibility, fault tolerance</li> </ul>                                                      |

## • *Cloud Computing*

- A computing model where any computing infrastructure can run on the cloud
- Hardware & Software are provided as remote services
- Elastic: grows and shrinks based on the user's demand
- Example: Amazon EC2



THANK YOU

